02_PCA Principal Component Analysis

Anish

2026-05-08

1. What this notebook does

PCA serves two roles in this project:

1. **Visualisation** project customers onto PC1–PC2 to eyeball cluster structure later.
2. **Preprocessing** replace ~7 correlated input features with 2–3 orthogonal PC scores to feed into k-means / hclust in `03_clustering.Rmd`.

Output: `../data/customer_pc_scores.csv` containing `CustomerID + PC1..PCk` (k = 3 by default).

2. Load and validate

```
customer <- read_csv("../data/processed/customer_features.csv",
                     show_col_types = FALSE)

stopifnot(
  "CustomerID" %in% names(customer),
  !anyDuplicated(customer$CustomerID),
  nrow(customer) > 0
)

glimpse(customer)
```

```
## Rows: 4,334
## Columns: 29
## $ CustomerID          <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 1~
## $ Recency             <dbl> 326, 2, 75, 19, 310, 36, 204, 232, 214, 23, 33, 2,~
## $ Frequency           <dbl> 1, 7, 4, 1, 1, 7, 1, 1, 1, 3, 1, 2, 4, 3, 1, 10, 2~
## $ GrossMonetary       <dbl> 77183.60, 4310.00, 1437.24, 1457.55, 294.40, 1385.~
## $ TotalQuantity       <dbl> 74215, 2458, 2332, 630, 196, 526, 20, 530, 240, 15~
## $ UniqueProducts      <dbl> 1, 103, 21, 72, 16, 57, 4, 58, 13, 52, 131, 12, 21~
## $ FirstPurchase       <dttm> 2011-01-18 10:01:00, 2010-12-07 14:57:00, 2010-12~
## $ LastPurchase        <dttm> 2011-01-18 10:01:00, 2011-12-07 15:52:00, 2011-09~
## $ TenureDays          <dbl> 0, 365, 282, 0, 0, 260, 0, 0, 0, 302, 0, 149, 274,~
## $ SinglePurchase      <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0,~
## $ AvgUnitPrice        <dbl> 1.0400000, 1.7534581, 0.6163122, 2.3135714, 1.5020~
## $ AvgOrderValue       <dbl> 77183.6000, 615.7143, 359.3100, 1457.5500, 294.400~
```

```
## $ AvgOrderItems       <dbl> 74215.00000, 351.14286, 583.00000, 630.00000, 196.~
## $ ReturnInvoices      <dbl> 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 3, 0,~
## $ ReturnValueAbs      <dbl> 77183.60, 0.00, 0.00, 0.00, 0.00, 120.33, 0.00, 0.~
## $ Country             <chr> "United Kingdom", "Iceland", "Finland", "Italy", "~
## $ NetMonetary         <dbl> 0.00, 4310.00, 1437.24, 1457.55, 294.40, 1265.41, ~
## $ ReturnRate          <dbl> 0.5000000, 0.0000000, 0.0000000, 0.0000000, 0.0000~
## $ ReturnValueRate     <dbl> 1.000000000, 0.000000000, 0.000000000, 0.000000000~
## $ AvgUnitPrice_winsor <dbl> 1.0400000, 1.7534581, 0.6163122, 2.3135714, 1.5020~
## $ log_Recency         <dbl> 5.789960, 1.098612, 4.330733, 2.995732, 5.739793, ~
## $ log_Frequency       <dbl> 0.6931472, 2.0794415, 1.6094379, 0.6931472, 0.6931~
## $ log_GrossMonetary   <dbl> 11.253955, 8.368925, 7.271175, 7.285198, 5.688330,~
## $ log_TotalQuantity   <dbl> 11.214735, 7.807510, 7.754910, 6.447306, 5.283204,~
## $ log_UniqueProducts  <dbl> 0.6931472, 4.6443909, 3.0910425, 4.2904594, 2.8332~
## $ log_AvgOrderValue   <dbl> 11.253955, 6.424406, 5.886965, 7.285198, 5.688330,~
## $ log_NetMonetary     <dbl> 0.000000, 8.368925, 7.271175, 7.285198, 5.688330, ~
## $ Monetary            <dbl> 77183.60, 4310.00, 1437.24, 1457.55, 294.40, 1385.~
## $ log_Monetary        <dbl> 11.253955, 8.368925, 7.271175, 7.285198, 5.688330,~
```

3. Modelling matrix

The seven-feature input drops everything that the EDA flagged as redundant or distortive:

- `log_NetMonetary` instead of `log_GrossMonetary` captures the customer's actual contribution to revenue net of returns.
- `AvgUnitPrice_winsor` instead of raw `AvgUnitPrice` without winsorising, a few extreme items dominate PC2/PC3 loadings.
- `Country` excluded UK dominance would force a near-degenerate split.
- `ReturnRate` / `ReturnValueRate` held back as profiling variables (sparse, mostly zero).

```
feature_cols <- c("log_Recency", "log_Frequency", "log_NetMonetary",
                  "log_UniqueProducts", "log_AvgOrderValue",
                  "AvgUnitPrice_winsor", "TenureDays")

X <- customer |>
  select(CustomerID, all_of(feature_cols)) |>
  drop_na()

# Sanity-check: row count must equal customer count.
stopifnot(nrow(X) == nrow(customer))

# Confirm winsorisation worked   pre-winsor max can be hundreds of pounds.
summary(X$AvgUnitPrice_winsor)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.08562 1.39329 1.78295 2.02562 2.31495 7.60380
```

```
quantile(X$AvgUnitPrice_winsor, c(.5, .9, .99, 1))
```

```
##      50%      90%      99%     100%
## 1.782945 3.210570 7.572846 7.603800
```

```r
dim(X)
```

```
## [1] 4334    8
```

```r
summary(X[, feature_cols])
```

```
##   log_Recency     log_Frequency    log_NetMonetary  log_UniqueProducts
##  Min.   :0.6931   Min.   :0.6931   Min.   :-4.959   Min.   :0.6931
##  1st Qu.:2.9444   1st Qu.:0.6931   1st Qu.: 5.696   1st Qu.:2.8332
##  Median :3.9512   Median :1.0986   Median : 6.474   Median :3.5835
##  Mean   :3.8321   Mean   :1.3416   Mean   : 6.532   Mean   :3.5496
##  3rd Qu.:4.9698   3rd Qu.:1.7918   3rd Qu.: 7.379   3rd Qu.:4.3567
##  Max.   :5.9269   Max.   :5.3327   Max.   :12.538   Max.   :7.4877
##  log_AvgOrderValue AvgUnitPrice_winsor   TenureDays
##  Min.   : 1.558    Min.   :0.08562     Min.   :  0.0
##  1st Qu.: 5.183    1st Qu.:1.39329     1st Qu.:  0.0
##  Median : 5.672    Median :1.78295     Median : 92.0
##  Mean   : 5.638    Mean   :2.02562     Mean   :130.3
##  3rd Qu.: 6.051    3rd Qu.:2.31495     3rd Qu.:252.0
##  Max.   :11.341    Max.   :7.60380     Max.   :373.0
```

4. Fit PCA

`prcomp(center = TRUE, scale. = TRUE)` standardises every column to zero mean / unit variance required because `AvgUnitPrice_winsor` and `TenureDays` are on totally different scales from the log features.

```r
pr.out <- prcomp(X[, feature_cols], center = TRUE, scale. = TRUE)
pr.out
```

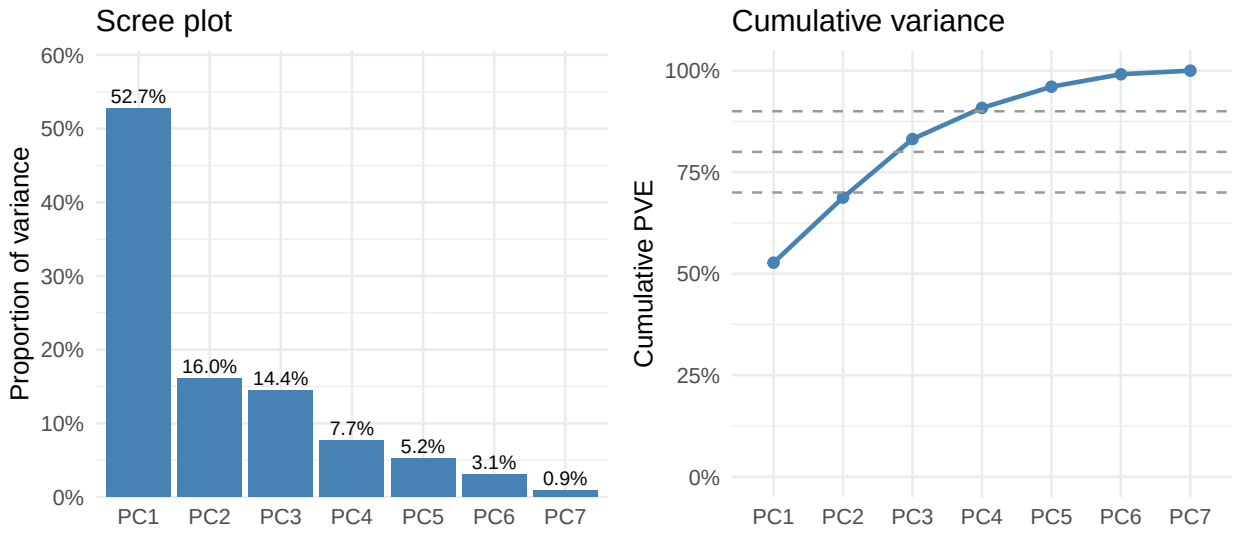
```
## Standard deviations (1, .., p=7):
## [1] 1.9208932 1.0595938 1.0043846 0.7323402 0.6042268 0.4637281 0.2493717
##
## Rotation (n x k) = (7 x 7):
##                             PC1        PC2         PC3         PC4         PC5
## log_Recency         -0.3543127 -0.3247000 -0.11967389 -0.86652203 -0.05794226
## log_Frequency        0.4588637  0.2848157 -0.08419794 -0.25300531 -0.16465563
## log_NetMonetary      0.4729638 -0.2306870 -0.18640031 -0.07994676 -0.22244905
## log_UniqueProducts   0.4243305 -0.1557292  0.14121516 -0.19256758  0.85444813
## log_AvgOrderValue    0.2472991 -0.7649580 -0.22875620  0.23764198 -0.19657842
## AvgUnitPrice_winsor -0.1137232  0.2058930 -0.93179092  0.08002433  0.26238229
## TenureDays           0.4313532  0.3292859 -0.05777853 -0.28067156 -0.28719068
##                             PC6          PC7
## log_Recency          0.008231418 -0.020618134
## log_Frequency        0.528447887 -0.575199481
## log_NetMonetary      0.352382528  0.712948214
## log_UniqueProducts  -0.092567368 -0.004207856
## log_AvgOrderValue   -0.219253920 -0.397698340
## AvgUnitPrice_winsor -0.033990784  0.006087152
## TenureDays          -0.733965432  0.046974079
```

5. Variance explained

```
pve <- pr.out$sdev^2 / sum(pr.out$sdev^2)
pve_df <- tibble(
  PC         = factor(paste0("PC", seq_along(pve)),
                      levels = paste0("PC", seq_along(pve))),
  PVE        = pve,
  cumulative = cumsum(pve)
)
pve_df
```

```
## # A tibble: 7 x 3
##   PC        PVE cumulative
##   <fct>   <dbl>      <dbl>
## 1 PC1   0.527        0.527
## 2 PC2   0.160        0.688
## 3 PC3   0.144        0.832
## 4 PC4   0.0766       0.908
## 5 PC5   0.0522       0.960
## 6 PC6   0.0307       0.991
## 7 PC7   0.00888      1
```

```
p_scree <- ggplot(pve_df, aes(PC, PVE)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = percent(PVE, accuracy = 0.1)),
            vjust = -0.4, size = 3) +
  scale_y_continuous(labels = percent_format(),
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Scree plot", x = NULL, y = "Proportion of variance")

p_cum <- ggplot(pve_df, aes(PC, cumulative, group = 1)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2) +
  geom_hline(yintercept = c(0.7, 0.8, 0.9),
             linetype = "dashed", colour = "grey60") +
  scale_y_continuous(labels = percent_format(), limits = c(0, 1)) +
  labs(title = "Cumulative variance", x = NULL, y = "Cumulative PVE")

gridExtra::grid.arrange(p_scree, p_cum, nrow = 1)
```




```
tibble(
  PC          = paste0("PC", seq_along(pr.out$sdev)),
  eigenvalue  = pr.out$sdev^2,
  keep_kaiser = pr.out$sdev^2 > 1
)
```

```
## # A tibble: 7 x 3
##   PC    eigenvalue keep_kaiser
##   <chr>      <dbl> <lgl>
## 1 PC1       3.69   TRUE
## 2 PC2       1.12   TRUE
## 3 PC3       1.01   TRUE
## 4 PC4       0.536  FALSE
## 5 PC5       0.365  FALSE
## 6 PC6       0.215  FALSE
## 7 PC7       0.0622 FALSE
```

Kaiser's rule (eigenvalue > 1) gives a quick orthogonal check on the elbow / variance-threshold heuristic. We typically keep the first three PCs across all three rules.

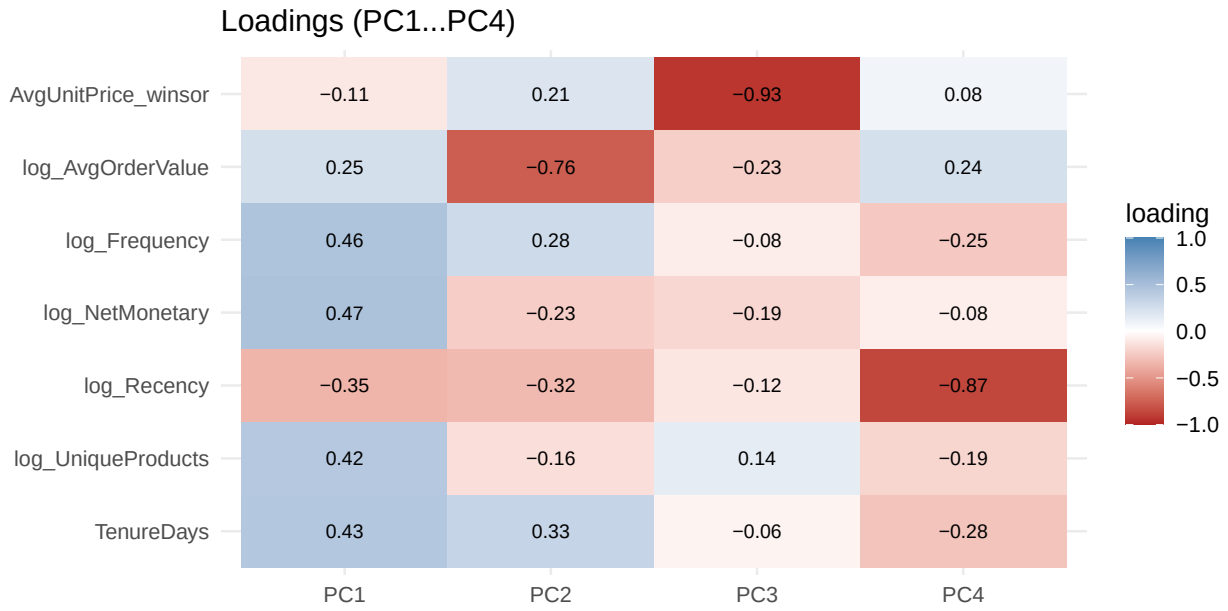
6. Loadings what each PC actually means

```
loadings <- as_tibble(pr.out$rotation, rownames = "feature")
loadings |> mutate(across(where(is.numeric), \(x) round(x, 2)))
```

```
## # A tibble: 7 x 8
##   feature              PC1   PC2   PC3   PC4   PC5   PC6   PC7
##   <chr>              <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 log_Recency        -0.35 -0.32 -0.12 -0.87 -0.06  0.01 -0.02
## 2 log_Frequency       0.46  0.28 -0.08 -0.25 -0.16  0.53 -0.58
## 3 log_NetMonetary     0.47 -0.23 -0.19 -0.08 -0.22  0.35  0.71
## 4 log_UniqueProducts  0.42 -0.16  0.14 -0.19  0.85 -0.09  0
```

```
## 5 log_AvgOrderValue    0.25 -0.76 -0.23  0.24 -0.2  -0.22 -0.4
## 6 AvgUnitPrice_winsor -0.11  0.21 -0.93  0.08  0.26 -0.03  0.01
## 7 TenureDays           0.43  0.33 -0.06 -0.28 -0.29 -0.73  0.05
```

```r
loadings |>
  pivot_longer(-feature, names_to = "PC", values_to = "loading") |>
  filter(PC %in% paste0("PC", 1:4)) |>
  mutate(PC = factor(PC, levels = paste0("PC", 1:4))) |>
  ggplot(aes(PC, fct_rev(feature), fill = loading)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", loading)), size = 3) +
  scale_fill_gradient2(low = "firebrick", mid = "white",
                       high = "steelblue", midpoint = 0,
                       limits = c(-1, 1)) +
  labs(title = "Loadings (PC1-PC4)", x = NULL, y = NULL)
```



Note on signs. PCA component signs are arbitrary `prcomp` could have flipped any column. What matters is the *contrast* across features, not the absolute sign. Read each PC as "high vs. low" along the loadings displayed below.

Reading the loadings on the cleaned/winsorised features:

- **PC1 engagement / customer value.** Loads positively on `log_Frequency`, `log_NetMonetary`, `log_UniqueProducts`, and `TenureDays`, and negatively on `log_Recency`. High PC1 = recent, frequent, broad-product, repeat-relationship customers. Low PC1 = lapsed one-time buyers.
- **PC2 order-size vs. tenure contrast.** Strong negative loading on `log_AvgOrderValue` and `AvgUnitPrice_winsor`, positive on `TenureDays` and `log_Frequency`. Low PC2 ~ wholesale-leaning behaviour (fewer, larger orders of higher-priced items). High PC2 ~ smaller, more frequent transactions over a longer relationship. **It is *not* purely a "tenure / lifecycle" axis** describing it that way ignores the AvgOrderValue contrast, which is the more informative half.
- **PC3 residual price-point.** Once PC1 and PC2 absorb engagement and order-size, PC3 picks up what's left of `AvgUnitPrice_winsor`
  essentially the price tier the customer typically buys at.

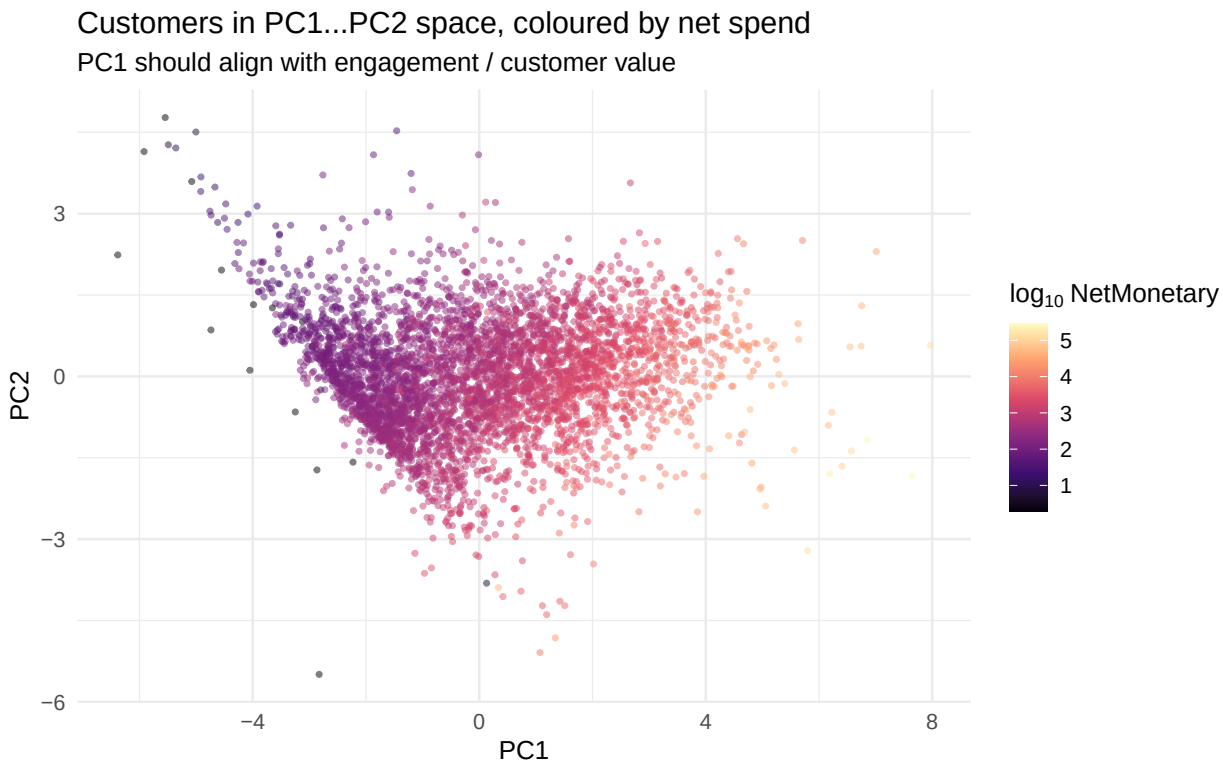
7. PC scores the new feature space

```
scores <- as_tibble(pr.out$x) |>
  mutate(CustomerID = X$CustomerID, .before = 1) |>
  left_join(customer |> select(CustomerID, GrossMonetary, NetMonetary,
                               Frequency, Recency, Country,
                               ReturnRate, ReturnValueRate, SinglePurchase),
            by = "CustomerID")

stopifnot(nrow(scores) == nrow(X), !anyDuplicated(scores$CustomerID))
head(scores)
```

```
## # A tibble: 6 x 16
##   CustomerID    PC1    PC2     PC3    PC4     PC5     PC6     PC7 GrossMonetary
##        <dbl>  <dbl>  <dbl>   <dbl>  <dbl>   <dbl>   <dbl>   <dbl>         <dbl>
## 1      12346 -2.83  -5.49  -0.371   1.86  -2.41   -2.91   -6.06          77184.
## 2      12347  3.34   0.224 -0.0869  0.928 -0.327  -0.571   0.0716         4310
## 3      12348  0.864 -0.214  0.800  -0.733 -1.29   -0.427   0.0816         1437.
## 4      12349  0.426 -2.27  -0.551   1.43   0.540  -0.137   0.0393         1458.
## 5      12350 -1.87  -0.960  0.421  -0.564 -0.177   0.0671 -0.0123          294.
## 6      12352  1.21   0.976 -0.555  -0.597  0.0665  0.0549 -0.0555         1386.
## # i 7 more variables: NetMonetary <dbl>, Frequency <dbl>, Recency <dbl>,
## #   Country <chr>, ReturnRate <dbl>, ReturnValueRate <dbl>,
## #   SinglePurchase <dbl>
```

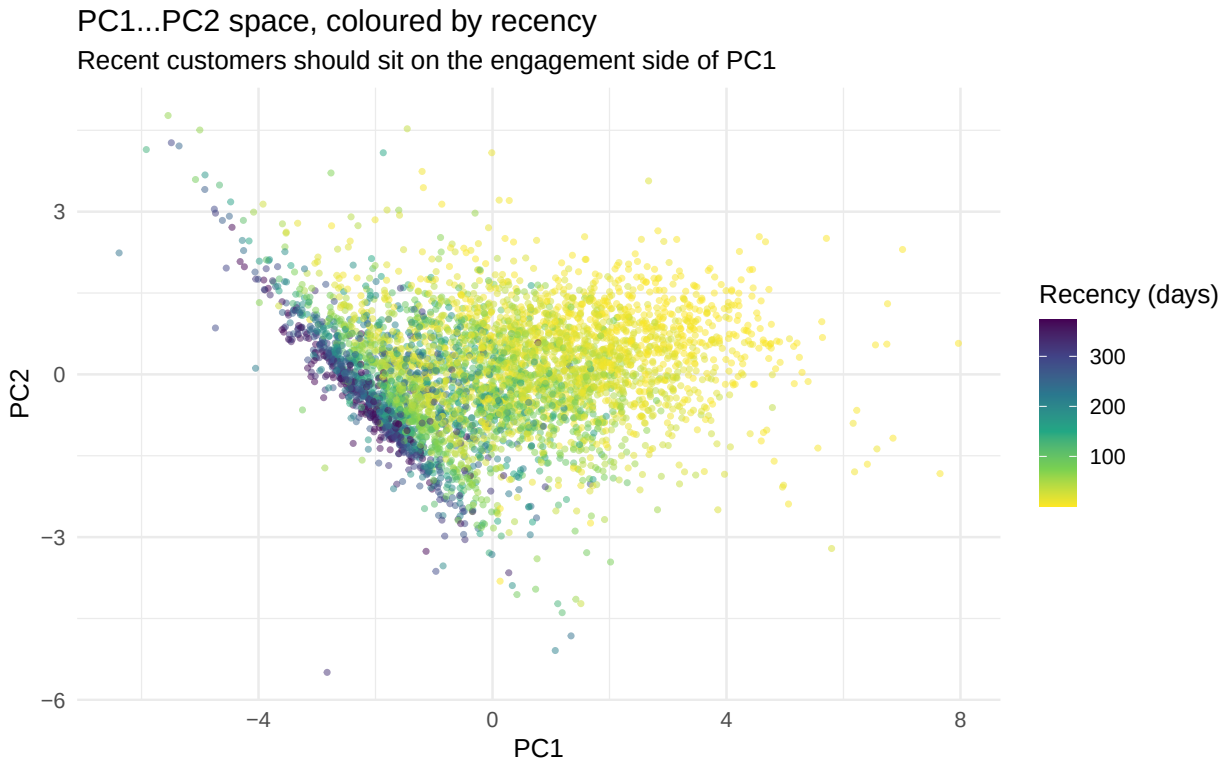
7a. PC1 vs PC2, coloured by net spend

```
ggplot(scores, aes(PC1, PC2,
                   colour = log10(pmax(NetMonetary, 1) + 1))) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_viridis_c(option = "magma",
                         name = expression(log[10]~NetMonetary)) +
  labs(title = "Customers in PC1-PC2 space, coloured by net spend",
       subtitle = "PC1 should align with engagement / customer value")
```



7b. PC1 vs PC2, coloured by recency

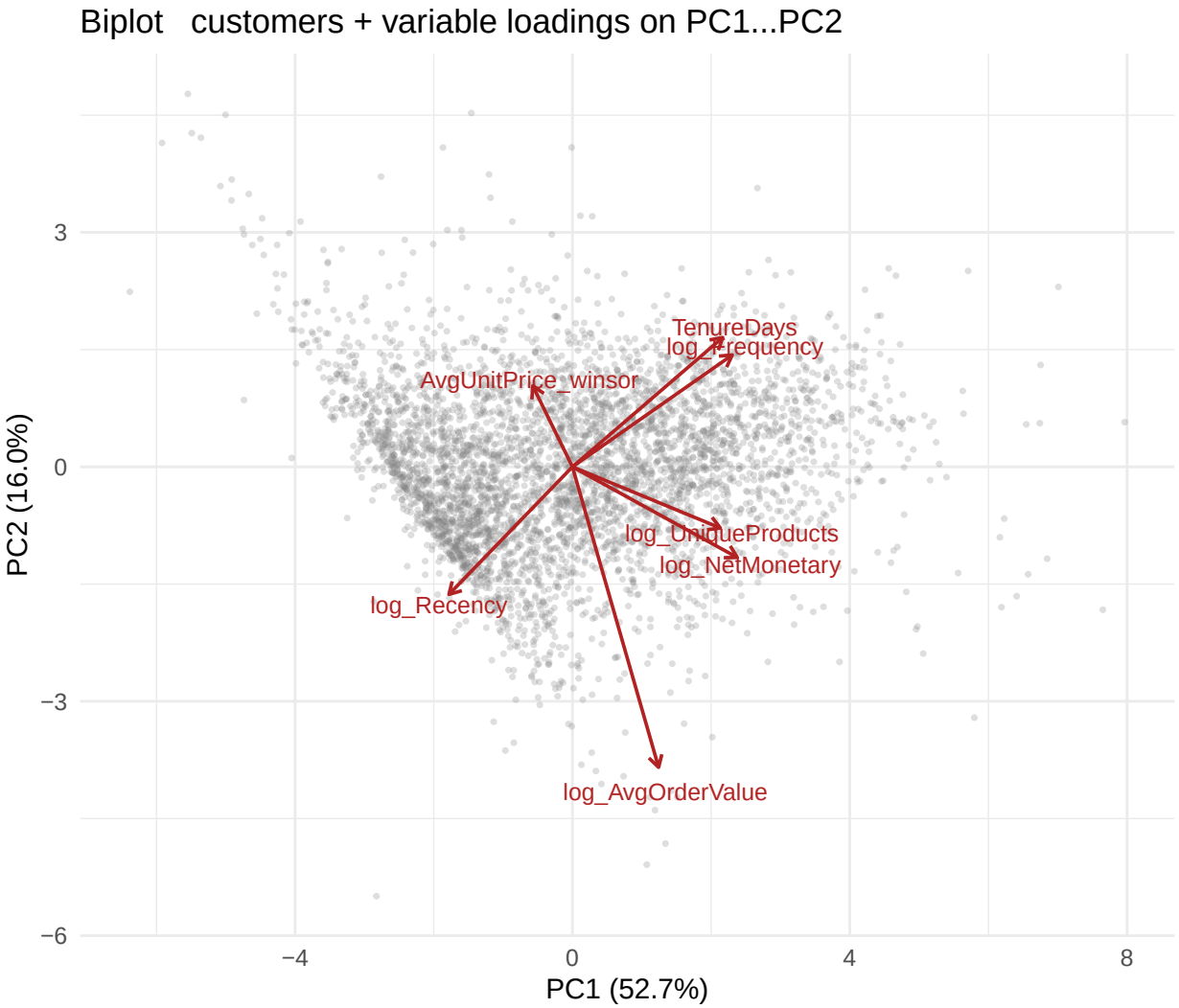
```
ggplot(scores, aes(PC1, PC2, colour = Recency)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_viridis_c(option = "viridis", direction = -1,
                         name = "Recency (days)") +
  labs(title = "PC1-PC2 space, coloured by recency",
       subtitle = "Recent customers should sit on the engagement side of PC1")
```



8. Biplot

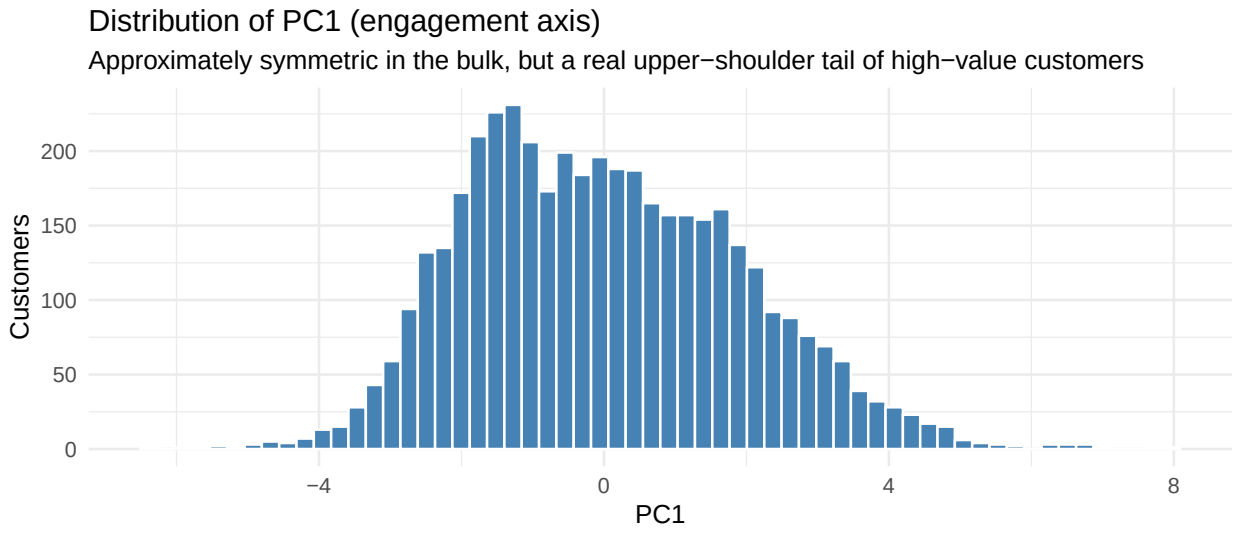
A ggplot biplot reads more clearly than `biplot(pr.out)` for the report.

```
arrow_scale <- with(pr.out,
                    min(max(abs(x[, 1])) / max(abs(rotation[, 1])),
                        max(abs(x[, 2])) / max(abs(rotation[, 2])))) * 0.7

arrows_df <- as_tibble(pr.out$rotation, rownames = "feature") |>
  mutate(PC1 = PC1 * arrow_scale,
         PC2 = PC2 * arrow_scale)

ggplot(scores, aes(PC1, PC2)) +
  geom_point(alpha = 0.25, size = 0.6, colour = "grey50") +
  geom_segment(data = arrows_df,
               aes(x = 0, y = 0, xend = PC1, yend = PC2),
               arrow = arrow(length = unit(0.2, "cm")),
               colour = "firebrick", linewidth = 0.7) +
  geom_text(data = arrows_df,
            aes(x = PC1 * 1.08, y = PC2 * 1.08, label = feature),
            colour = "firebrick", size = 3.5) +
  labs(title = "Biplot   customers + variable loadings on PC1-PC2",
       x = sprintf("PC1 (%s)", percent(pve[1], accuracy = 0.1)),
       y = sprintf("PC2 (%s)", percent(pve[2], accuracy = 0.1)))
```



9. PC1 sanity check symmetric core, real upper tail

```
ggplot(scores, aes(PC1)) +
  geom_histogram(bins = 60, fill = "steelblue", colour = "white") +
  labs(title = "Distribution of PC1 (engagement axis)",
       subtitle = paste("Approximately symmetric in the bulk,",
                        "but a real upper-shoulder tail of high-value customers"),
       x = "PC1", y = "Customers")
```



PC1 is approximately symmetric in the bulk but retains a right shoulder representing the wholesale-leaning tail. We do **not** claim the distribution is "well-behaved enough" for k-means to be unconditionally appropriate; we instead carry this through as a known sensitivity and report the segmentation on both raw and unit-variance-scaled PC scores in `03_clustering.Rmd`.

```
quantile(scores$PC1,
         probs = c(0.5, 0.75, 0.9, 0.95, 0.99, 1.0)) |>
  round(2)
```

```
##   50%   75%   90%   95%   99%  100%
## -0.16  1.37  2.61  3.34  4.66  7.97
```

The gap between the 95th and 99th percentile (and again 99th to max) is where the wholesale shoulder lives those rows will end up in the high-value cluster in `03_clustering.Rmd` regardless of `k`, but they also pull the centroid of that cluster, which is why the `04_profiles.Rmd` table reports both the median and the mean.